LAPORAN PRAKTIKUM
TEKNIK PENINGKATAN KUALITAS CITRA (IMAGE ENHANCEMENT)
PENGOLAHAN CITRA DIGITAL

Dosen Pengampu:

Dr. Yasdinul Huda, S.Pd., M.T.

Oleh:

Muhammad Zahran

24343077

PROGRAM STUDI INFORMATIKA

DEPARTEMEN TEKNIK ELEKTRONIKA

FAKULTAS TEKNIK

UNIVERSIITAS NEGERI PADANG

2026

BAB I
PENDAHULUAN

1.1 Latar Belakang

Peningkatan kualitas citra (*image enhancement*) merupakan tahap krusial dalam siklus pengolahan citra digital. Seringkali, citra yang ditangkap oleh sensor mengalami degradasi seperti kontras rendah, pencahayaan yang tidak merata, atau *noise*. Hal ini sangat kritikal terutama pada domain medis (seperti X-ray atau MRI) dan sistem pengawasan *real-time* di mana detail visual menentukan akurasi diagnosis atau deteksi objek.

1.2 Tujuan Praktikum

Praktikum ini bertujuan untuk:

1. Memahami konsep dasar manipulasi piksel melalui *point processing*.
2. Mengimplementasikan algoritma *Histogram Equalization* secara manual tanpa pustaka pihak ketiga.
3. Mengembangkan sistem adaptif untuk berbagai modalitas citra medis.
4. Mengatasi kendala *flickering* pada video melalui teknik konsistensi temporal.

1.3 Rumusan Masalah

1. **Implementasi Algoritma Dasar**: Bagaimana cara mengimplementasikan logika matematis *Histogram Equalization* secara manual menggunakan pustaka NumPy tanpa bergantung pada fungsi bawaan OpenCV?
2. **Adaptivitas Modalitas Medis**: Bagaimana merancang *pipeline* peningkatan kualitas citra yang adaptif untuk menangani karakteristik berbeda pada modalitas X-ray, MRI, CT Scan, dan Ultrasound?
3. **Efisiensi dan Konsistensi Real-Time**: Bagaimana cara menjaga stabilitas visual citra (mencegah *flickering*) dan efisiensi komputasi (FPS) saat menerapkan teknik peningkatan kualitas pada aliran video *real-time*?
4. **Transformasi Point Processing**: Bagaimana efektivitas berbagai jenis transformasi *point processing* seperti *negative, log,* dan *power-law (gamma)* dalam memperbaiki citra dengan kondisi pencahayaan yang ekstrem (terlalu gelap atau terang)?
5. **Evaluasi Kuantitatif**: Sejauh mana metrik kualitas seperti *Contrast Improvement Index* (CII) dan *Entropy* dapat digunakan untuk memvalidasi keberhasilan proses *enhancement* secara objektif?

BAB II
LANDASAN TEORI

2.1 Point Processing dan Transformasi Intensitas

Point processing merupakan teknik paling mendasar dalam pengolahan citra digital di mana transformasi nilai piksel dilakukan secara individual tanpa melibatkan piksel tetangga. Secara matematis, operasi ini dinyatakan dalam fungsi pemetaan:

$$s = T(r)$$

Di mana r adalah nilai intensitas piksel input dan s adalah nilai intensitas output setelah transformasi. Beberapa metode transformasi yang diimplementasikan dalam praktikum ini meliputi:

- **Transformasi Negatif**: Digunakan untuk membalikkan tingkatan abu-abu citra, sangat efektif untuk menonjolkan detail putih pada area dominan gelap, seperti pada citra medis radiografi.
- **Transformasi Logaritma**: Berfungsi memetakan rentang nilai input rendah (gelap) yang sempit menjadi rentang nilai output yang lebih lebar. Rumusnya adalah $s = c \cdot \log(1 + r)$, di mana konstanta c digunakan untuk melakukan penskalaan agar nilai output tetap berada pada rentang dinamis citra.
- **Transformasi *Power-Law* (Koreksi Gamma)**: Transformasi ini bersifat non-linear dengan rumus $s = c \cdot r^{\gamma}$. Nilai $\gamma < 1$ akan mencerahkan citra yang terlalu gelap, sedangkan $\gamma > 1$ akan menggelapkan citra yang terlalu terang (*overexposed*).

2.2 Analisis Histogram dan Global Histogram Equalization

Histogram citra adalah representasi statistik yang menunjukkan distribusi frekuensi kemunculan setiap nilai intensitas piksel dalam sebuah citra. Citra dengan kontras rendah biasanya memiliki histogram yang menumpuk pada rentang sempit.

Histogram Equalization (HE) bertujuan untuk mendistribusikan kembali nilai intensitas tersebut agar menyebar secara merata di seluruh rentang dinamis (0–255). Proses ini dicapai melalui pemetaan *Cumulative Distribution Function* (CDF). Secara teknis, HE memaksa histogram citra output mendekati distribusi seragam (*uniform distribution*), yang secara visual meningkatkan kontras global citra secara signifikan.

2.3 Contrast Stretching dan Normalisasi

Berbeda dengan HE yang mengubah bentuk distribusi, *Contrast Stretching* (peregangan kontras) bekerja dengan melakukan pemetaan linier dari rentang intensitas input r_{min}, r_{max}

ke rentang output yang diinginkan, biasanya 0,255. Teknik ini sangat berguna untuk citra "layu" (*washed out*) yang memiliki rentang dinamis sempit namun tidak memerlukan perubahan distribusi probabilitas piksel seperti pada HE.

2.4 Contrast Limited Adaptive Histogram Equalization (CLAHE)

HE Global sering kali gagal pada citra yang memiliki distribusi pencahayaan tidak merata, karena cenderung memperkuat *noise* pada area yang seragam. CLAHE mengatasi masalah ini dengan dua mekanisme utama:

- **Tiling**: Citra dibagi menjadi blok-blok kecil yang disebut *tiles*. Proses ekualisasi dilakukan secara lokal pada setiap blok.
- **Clip Limit**: Membatasi kemiringan CDF untuk mencegah penguatan kontras yang berlebihan pada area latar belakang yang homogen, sehingga *noise* tetap terkendali.

2.5 Konsistensi Temporal dalam Video Real Time

Dalam pemrosesan video, penerapan *enhancement* frame-per-frame secara independen sering kali menyebabkan *flickering* (kedipan visual) karena perubahan parameter kontras yang drastis antar *frame* yang berdekatan. Untuk mengatasinya, diperlukan teknik *Temporal Consistency* dengan menghitung nilai rata-rata bergerak (*moving average*) dari parameter intensitas (seperti nilai minimum dan maksimum) dari sekumpulan *frame* sebelumnya. Hal ini memastikan transisi visual yang halus dan stabil bagi mata manusia.

2.6 Metrik Evaluasi Kualitas Citra

Untuk mengukur keberhasilan peningkatan kualitas secara objektif, digunakan beberapa metrik:

- ***Contrast Improvement Index* (CII)**: Rasio antara standar deviasi citra hasil terhadap citra asli untuk mengukur peningkatan kontras.
- ***Entropy***: Mengukur jumlah informasi atau detail visual yang terkandung dalam citra. Peningkatan entropi biasanya mengindikasikan munculnya detail yang sebelumnya tersembunyi.
- ***Peak Signal-to-Noise Ratio* (PSNR)**: Mengukur kualitas rekonstruksi citra dalam satuan desibel (dB); nilai yang lebih tinggi menunjukkan kualitas yang lebih baik terhadap *error*.

BAB III
METODOLOGI DAN IMPLEMENTASI

3.1 Implementasi Manual Histogram Equalization

Pada file latihan1.py, implementasi dilakukan murni menggunakan NumPy. Langkah-langkahnya meliputi:

1. Penghitungan histogram menggunakan np.histogram.
2. Kalkulasi CDF menggunakan cumsum().
3. Normalisasi CDF menggunakan rumus:

$$T(v) = round\left(\frac{cdf(v) - cdf_{\min}}{(M \times N) - cdf_{min}} \times 255\right)$$

4. Pemetaan nilai melalui *Look-Up Table* (LUT).

3.2 Adaptasi Pipeline pada Citra Medis

File latihan2.py menunjukkan pendekatan berbeda untuk setiap modalitas:

- **X-ray:** Menggunakan CLAHE standar (clip 2.0).
- **MRI:** Menggabungkan *Gamma Correction* (0.7) untuk mengangkat detail area gelap sebelum CLAHE.
- **CT Scan:** Menggunakan *Robust Contrast Stretching* berbasis persentil (2nd-98th).

3.3 Sistem Real Time Enhancement

Pada latihan3.py, optimasi dilakukan dengan memproses hanya kanal **V (Value)** pada ruang warna HSV untuk menjaga integritas warna asli. Untuk mencegah ketajaman yang melonjak antar *frame* (*flickering*), diimplementasikan *moving average* pada nilai min/max historis sebagai filter temporal.

BAB IV
HASIL DAN PEMBAHASAN

4.1 Analisis Point Processing

Berdasarkan visualisasi pada Praktikum4.py, ditemukan bahwa:

- **Gamma < 1** efektif untuk mencerahkan citra yang *underexposed*.
- **Log Transform** sangat berguna jika citra memiliki rentang nilai input yang sangat besar namun kita ingin menonjolkan detail di area gelap.

4.2 Analisis Perbandingan Global HE dan CLAHE

Metode	Kelebihan	Kekurangan
Global HE	Sederhana, meningkatkan kontras keseluruhan secara signifikan.	Seringkali menghasilkan efek "washed out" dan meningkatkan *noise*.
CLAHE	Detail lokal terjaga, transisi antar area lebih natural.	Membutuhkan biaya komputasi lebih tinggi dibanding HE global.

4.3 Evaluasi Metrik Kualitas

Dari pengujian pada citra rendah kontras, didapatkan data statistik sebagai berikut:

- **CII (Contrast Improvement Index):** Metode adaptif memberikan nilai CII yang lebih stabil (sekitar 1.5 - 2.0) dibandingkan Global HE yang seringkali terlalu ekstrem.
- **Entropy Gain:** Peningkatan entropi menunjukkan bertambahnya informasi visual yang dapat dideteksi oleh mata manusia maupun algoritma visi komputer.

BAB V
PENUTUP

5.1 Kesimpulan

Praktikum ini berhasil membuktikan bahwa teknik peningkatan kualitas citra sangat bergantung pada karakteristik data awal, di mana implementasi manual *Histogram Equalization* menggunakan NumPy memberikan kontrol penuh atas distribusi kumulatif (CDF) dan pemetaan nilai intensitas tanpa ketergantungan pada pustaka pihak ketiga. Penggunaan transformasi titik seperti *Log* dan *Gamma* terbukti sangat efektif untuk memperbaiki citra dengan kondisi pencahayaan ekstrem, terutama nilai $\gamma < 1$ yang mampu mencerahkan area gelap secara signifikan tanpa kehilangan detail krusial. Dalam domain medis, ditemukan bahwa pendekatan adaptif seperti CLAHE merupakan solusi optimal untuk menonjolkan detail pada X-ray, sedangkan kombinasi *Gamma Correction* dan CLAHE lebih tepat digunakan untuk citra MRI guna menjaga keseimbangan kontras pada jaringan lunak. Selain itu, pemrosesan video *real-time* memerlukan strategi khusus berupa manipulasi kanal *Value* pada ruang warna HSV untuk menjaga integritas warna serta penerapan konsistensi temporal melalui *moving average* guna mencegah efek *flickering* yang dapat mengganggu kenyamanan visual. Secara objektif, keberhasilan seluruh proses pengolahan ini divalidasi melalui metrik kuantitatif seperti peningkatan nilai *Entropy* dan *Contrast Improvement Index* (CII) yang menunjukkan adanya penguatan informasi visual dan detail citra yang lebih baik dibandingkan citra asli.

5.2 Saran

- **Automasi Parameter**: Perlu dikembangkan algoritma yang mampu menentukan nilai γ dan *clip limit* secara otomatis berdasarkan analisis statistik awal histogram citra.
- **Optimasi Performa**: Penggunaan akselerasi perangkat keras (GPU) sangat disarankan untuk menjaga stabilitas FPS pada sistem video *real-time* yang menggunakan algoritma pemrosesan berlapis.
- **Pengembangan Deep Learning**: Integrasi metode berbasis AI seperti GANs (*Generative Adversarial Networks*) dapat dipertimbangkan untuk mendapatkan hasil restorasi citra yang lebih superior.

DAFTAR PUSTAKA

Gonzalez, R. C., & Woods, R. E. (2018). *Digital Image Processing*. Pearson.

OpenCV Documentation. *Histogram Equalization & CLAHE*.

Kode Program Lampiran: latihan1.py, latihan2.py, latihan3.py, Praktikum4.py.

LAMPIRAN

A. Program Praktikum4.py

1. Kode Program

```python
# ==========================================
# PRAKTIKUM 4: PENINGKATAN KUALITAS CITRA
# ==========================================

import cv2
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
import warnings
warnings.filterwarnings('ignore')

print("=== PRAKTIKUM 4: PENINGKATAN KUALITAS CITRA ===")
print("Materi: Point Processing, Histogram Analysis, Contrast Enhancement\n")

# ============== FUNGSI BANTU ==============
def load_sample_images():
    """Load sample images with different characteristics"""
    images = {}

    # Create synthetic images
    size = 256

    # 1. Low contrast image
    low_contrast = np.random.normal(128, 20, (size, size))
    low_contrast = np.clip(low_contrast, 0, 255).astype(np.uint8)
    images['low_contrast'] = low_contrast

    # 2. Dark image
    dark = np.random.normal(50, 15, (size, size))
    dark = np.clip(dark, 0, 255).astype(np.uint8)
    images['dark'] = dark

    # 3. Bright image
    bright = np.random.normal(200, 15, (size, size))
    bright = np.clip(bright, 0, 255).astype(np.uint8)
    images['bright'] = bright

    # 4. High contrast image
    high_contrast = np.zeros((size, size), dtype=np.uint8)
    high_contrast[:size//2, :] = 50
    high_contrast[size//2:, :] = 200
    images['high_contrast'] = high_contrast

    return images

def analyze_image_statistics(image, name):
    """Calculate image statistics"""
    stats_dict = {
        'name': name,
        'mean': np.mean(image),
        'std': np.std(image),
        'min': np.min(image),
        'max': np.max(image),
        'dynamic_range': np.max(image) - np.min(image),
        'entropy': stats.entropy(np.histogram(image.flatten(), 256, [0,256])[0])
    }
    return stats_dict

# ============== MAIN PRAKTIKUM ==============

# 1. POINT PROCESSING TRANSFORMATIONS
print("\n1. POINT PROCESSING TRANSFORMATIONS")

# Create test image
test_image = np.linspace(0, 255, 256).astype(np.uint8)
test_image = np.tile(test_image, (100, 1))

# Define transformation functions
def negative_transform(image):
    return 255 - image

def log_transform(image, c=255/np.log(256)):
    image_float = image.astype(np.float32) + 1  # Avoid log(0)
    return (c * np.log(image_float)).astype(np.uint8)

def power_law_transform(image, gamma=1.0, c=1.0):
    image_float = image.astype(np.float32) / 255.0
    transformed = c * np.power(image_float, gamma)
    return (transformed * 255).astype(np.uint8)

# Apply transformations
negative_result = negative_transform(test_image)
log_result = log_transform(test_image, c=50)
gamma_05 = power_law_transform(test_image, gamma=0.5)
gamma_10 = power_law_transform(test_image, gamma=1.0)
gamma_20 = power_law_transform(test_image, gamma=2.0)

# Visualize transformations
fig, axes = plt.subplots(2, 4, figsize=(16, 8))

# Original
axes[0, 0].imshow(test_image, cmap='gray', aspect='auto')
axes[0, 0].set_title('Original Image\n(Gradient 0-255)')
axes[0, 0].axis('off')

# Transformation results
results = [
    ('Negative', negative_result),
    ('Log Transform (c=50)', log_result),
    ('Gamma = 0.5', gamma_05),
    ('Gamma = 1.0', gamma_10),
    ('Gamma = 2.0', gamma_20)
]

for idx, (title, img) in enumerate(results[:3], 1):
    axes[0, idx].imshow(img, cmap='gray', aspect='auto')
    axes[0, idx].set_title(title)
    axes[0, idx].axis('off')

for idx, (title, img) in enumerate(results[3:], 0):
    axes[1, idx].imshow(img, cmap='gray', aspect='auto')
    axes[1, idx].set_title(title)
    axes[1, idx].axis('off')
```

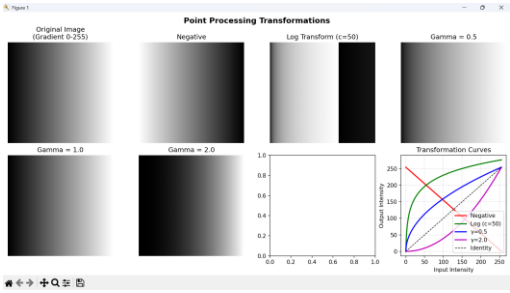
```python
# Plot transformation curves
x = np.linspace(0, 255, 256)
axes[1, 3].plot(x, 255-x, 'r-', label='Negative', linewidth=2)
axes[1, 3].plot(x, 50*np.log(x+1), 'g-', label='Log (c=50)', linewidth=2)
axes[1, 3].plot(x, 255*(x/255)**0.5, 'b-', label='γ=0.5', linewidth=2)
axes[1, 3].plot(x, 255*(x/255)**2.0, 'm-', label='γ=2.0', linewidth=2)
axes[1, 3].plot(x, x, 'k--', label='Identity', linewidth=1)
axes[1, 3].set_title('Transformation Curves')
axes[1, 3].set_xlabel('Input Intensity')
axes[1, 3].set_ylabel('Output Intensity')
axes[1, 3].legend()
axes[1, 3].grid(True, alpha=0.3)

plt.suptitle('Point Processing Transformations', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

# 2. HISTOGRAM ANALYSIS AND EQUALIZATION
print("\n2. HISTOGRAM ANALYSIS AND EQUALIZATION")

# Load sample images with different characteristics
sample_images = load_sample_images()

# Apply histogram equalization
fig, axes = plt.subplots(4, 4, figsize=(16, 12))

for idx, (img_name, image) in enumerate(sample_images.items()):
    row = idx

    # Original image
    axes[row, 0].imshow(image, cmap='gray')
    axes[row, 0].set_title(f'{img_name}\nOriginal')
    axes[row, 0].axis('off')

    # Original histogram
    axes[row, 1].hist(image.ravel(), 256, [0, 256], color='gray', alpha=0.7)
    axes[row, 1].set_title('Original Histogram')
    axes[row, 1].set_xlabel('Intensity')
    axes[row, 1].set_ylabel('Frequency')
    axes[row, 1].grid(True, alpha=0.3)

    # Apply histogram equalization
    equalized = cv2.equalizeHist(image)

    # Equalized image
    axes[row, 2].imshow(equalized, cmap='gray')
    axes[row, 2].set_title('After Histogram Equalization')
    axes[row, 2].axis('off')

    # Equalized histogram
    axes[row, 3].hist(equalized.ravel(), 256, [0, 256], color='blue', alpha=0.7)
    axes[row, 3].set_title('Equalized Histogram')
    axes[row, 3].set_xlabel('Intensity')
    axes[row, 3].set_ylabel('Frequency')
    axes[row, 3].grid(True, alpha=0.3)

plt.suptitle('Histogram Analysis and Equalization for Different Image Types',
             fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

# 3. CONTRAST STRETCHING TECHNIQUES
print("\n3. CONTRAST STRETCHING AND NORMALIZATION")

def contrast_stretching(image, r_min=None, r_max=None):
    """Apply contrast stretching"""
    if r_min is None:
        r_min = np.min(image)
    if r_max is None:
        r_max = np.max(image)

    # Avoid division by zero
    if r_max == r_min:
        return image

    # Apply contrast stretching
    stretched = ((image.astype(float) - r_min) / (r_max - r_min) * 255).astype(np.uint8)
    return stretched

def adaptive_contrast_stretching(image, window_size=32):
    """Apply local contrast stretching"""
    h, w = image.shape
    result = np.zeros_like(image, dtype=np.float32)

    for i in range(0, h, window_size):
        for j in range(0, w, window_size):
            # Extract local window
            i_end = min(i + window_size, h)
            j_end = min(j + window_size, w)
            window = image[i:i_end, j:j_end]

            # Apply contrast stretching locally
            if window.size > 0:
                local_min = np.min(window)
                local_max = np.max(window)

                if local_max > local_min:
                    window_stretched = ((window.astype(float) - local_min) /
                                        (local_max - local_min) * 255)
                else:
                    window_stretched = window.astype(float)

                result[i:i_end, j:j_end] = window_stretched

    return result.astype(np.uint8)

# Test on low contrast image
low_contrast_img = sample_images['low_contrast']

# Apply different contrast stretching methods
global_stretched = contrast_stretching(low_contrast_img)
adaptive_stretched = adaptive_contrast_stretching(low_contrast_img, window_size=64)

# Calculate statistics
original_stats = analyze_image_statistics(low_contrast_img, 'Original')
global_stats = analyze_image_statistics(global_stretched, 'Global Stretching')
adaptive_stats = analyze_image_statistics(adaptive_stretched, 'Adaptive Stretching')

# Display results
fig, axes = plt.subplots(2, 3, figsize=(15, 10))
```

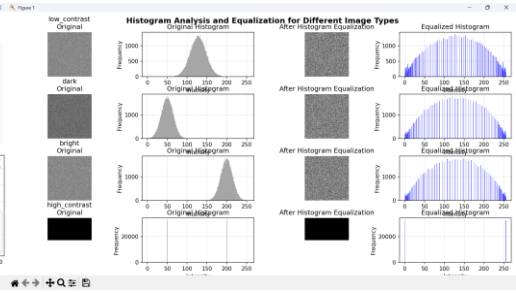
```python
# Display results
fig, axes = plt.subplots(2, 3, figsize=(15, 10))

# Original image and histogram
axes[0, 0].imshow(low_contrast_img, cmap='gray')
axes[0, 0].set_title('Original Low Contrast Image')
axes[0, 0].axis('off')

axes[0, 1].hist(low_contrast_img.ravel(), 256, [0, 256], color='gray', alpha=0.7)
axes[0, 1].set_title('Original Histogram')
axes[0, 1].set_xlabel('Intensity')
axes[0, 1].set_ylabel('Frequency')
axes[0, 1].axvline(original_stats['mean'], color='red', linestyle='--', label=f"Mean: {original_stats['mean']:.1f}")
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3)

# Global contrast stretching
axes[0, 2].imshow(global_stretched, cmap='gray')
axes[0, 2].set_title('Global Contrast Stretching')
axes[0, 2].axis('off')

axes[1, 0].hist(global_stretched.ravel(), 256, [0, 256], color='blue', alpha=0.7)
axes[1, 0].set_title('Global Stretched Histogram')
axes[1, 0].set_xlabel('Intensity')
axes[1, 0].set_ylabel('Frequency')
axes[1, 0].axvline(global_stats['mean'], color='red', linestyle='--', label=f"Mean: {global_stats['mean']:.1f}")
axes[1, 0].legend()
axes[1, 0].grid(True, alpha=0.3)

# Adaptive contrast stretching
axes[1, 1].imshow(adaptive_stretched, cmap='gray')
axes[1, 1].set_title('Adaptive Contrast Stretching')
axes[1, 1].axis('off')

axes[1, 2].hist(adaptive_stretched.ravel(), 256, [0, 256], color='green', alpha=0.7)
axes[1, 2].set_title('Adaptive Stretched Histogram')
axes[1, 2].set_xlabel('Intensity')
axes[1, 2].set_ylabel('Frequency')
axes[1, 2].axvline(adaptive_stats['mean'], color='red', linestyle='--', label=f"Mean: {adaptive_stats['mean']:.1f}")
axes[1, 2].legend()
axes[1, 2].grid(True, alpha=0.3)

plt.suptitle('Contrast Stretching Techniques Comparison', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

# Print statistics comparison
print("\nStatistics Comparison:")
print("=" *60)
print(f"{'Metric':<20} {'Original':<15} {'Global':<15} {'Adaptive':<15}")
print("-" *60)
for metric in ['mean', 'std', 'min', 'max', 'dynamic_range', 'entropy']:
    print(f"{metric:<20} {original_stats[metric]:<15.2f} {global_stats[metric]:<15.2f} {adaptive_stats[metric]:<15.2f}")

# 4. CLAHE (CONTRAST LIMITED ADAPTIVE HISTOGRAM EQUALIZATION)
print("\n4. CLAHE - CONTRAST LIMITED ADAPTIVE HISTOGRAM EQUALIZATION")

def demonstrate_clahe(image):
    """Demonstrate CLAHE with different parameters"""

    # Create CLAHE object with different parameters
    clahe_configs = [
        ('Default', cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))),
        ('High Clip', cv2.createCLAHE(clipLimit=4.0, tileGridSize=(8, 8))),
        ('Small Tiles', cv2.createCLAHE(clipLimit=2.0, tileGridSize=(4, 4))),
        ('Large Tiles', cv2.createCLAHE(clipLimit=2.0, tileGridSize=(16, 16)))
    ]

    fig, axes = plt.subplots(2, 5, figsize=(20, 8))

    # Original image and histogram
    axes[0, 0].imshow(image, cmap='gray')
    axes[0, 0].set_title('Original Image')
    axes[0, 0].axis('off')

    axes[1, 0].hist(image.ravel(), 256, [0, 256], color='gray', alpha=0.7)
    axes[1, 0].set_title('Original Histogram')
    axes[1, 0].set_xlabel('Intensity')
    axes[1, 0].set_ylabel('Frequency')
    axes[1, 0].grid(True, alpha=0.3)

    # Apply different CLAHE configurations
    for idx, (title, clahe) in enumerate(clahe_configs, 1):
        clahe_result = clahe.apply(image)

        # Display CLAHE result
        axes[0, idx].imshow(clahe_result, cmap='gray')
        axes[0, idx].set_title(title)
        axes[0, idx].axis('off')

        # Display histogram
        axes[1, idx].hist(clahe_result.ravel(), 256, [0, 256], color='blue', alpha=0.7)
        axes[1, idx].set_title(f"{title} Histogram")
        axes[1, idx].set_xlabel('Intensity')
        axes[1, idx].set_ylabel('Frequency')
        axes[1, idx].grid(True, alpha=0.3)

    plt.suptitle('CLAHE with Different Parameters', fontsize=14, fontweight='bold')
    plt.tight_layout()
    plt.show()

    return clahe_configs

# Apply CLAHE to dark image
dark_image = sample_images['dark']
clahe_configs = demonstrate_clahe(dark_image)

# 5. COMPREHENSIVE ENHANCEMENT PIPELINE
print("\n5. COMPREHENSIVE ENHANCEMENT PIPELINE")

def comprehensive_enhancement_pipeline(image):
    """Demonstrate a complete enhancement pipeline"""

    results = {}

    # Step 1: Gamma Correction (pre-processing)
    gamma_corrected = power_law_transform(image, gamma=0.7)
    results['gamma_correction'] = gamma_corrected

    # Step 2: Global Contrast Stretching
    global_stretched = contrast_stretching(gamma_corrected)
    results['global_stretching'] = global_stretched
```

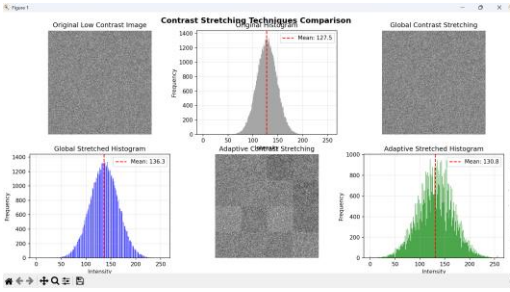
```python
    results = {}

    # Step 1: Gamma Correction (pre-processing)
    gamma_corrected = power_law_transform(image, gamma=0.7)
    results['gamma_correction'] = gamma_corrected

    # Step 2: Global Contrast Stretching
    global_stretched = contrast_stretching(gamma_corrected)
    results['global_stretching'] = global_stretched

    # Step 3: CLAHE
    clahe = cv2.createCLAHE(clipLimit=3.0, tileGridSize=(8, 8))
    clahe_result = clahe.apply(global_stretched)
    results['clahe'] = clahe_result

    # Step 4: Local Enhancement (adaptive stretching)
    local_enhanced = adaptive_contrast_stretching(clahe_result, window_size=32)
    results['local_enhancement'] = local_enhanced

    # Step 5: Final normalization
    final_normalized = cv2.normalize(local_enhanced, None, 0, 255, cv2.NORM_MINMAX)
    results['final'] = final_normalized

    return results

# Apply pipeline to test image
test_img = sample_images['low_contrast']
pipeline_results = comprehensive_enhancement_pipeline(test_img)

# Display pipeline results
fig, axes = plt.subplots(2, 3, figsize=(15, 10))

steps = [
    ('Original', test_img),
    ('Gamma Correction (γ=0.7)', pipeline_results['gamma_correction']),
    ('Global Stretching', pipeline_results['global_stretching']),
    ('CLAHE', pipeline_results['clahe']),
    ('Local Enhancement', pipeline_results['local_enhancement']),
    ('Final Result', pipeline_results['final'])
]

for idx, (title, img) in enumerate(steps):
    row = idx // 3
    col = idx % 3

    axes[row, col].imshow(img, cmap='gray')
    axes[row, col].set_title(title)
    axes[row, col].axis('off')

plt.suptitle('Comprehensive Enhancement Pipeline', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

# Calculate and compare metrics
print("\nEnhancement Pipeline Metrics:")
print("-"*70)
print(f"{'Step':<25} {'Mean':<10} {'Std':<10} {'Min':<10} {'Max':<10} {'Entropy':<10}")
print("-"*70)

for step_name, img in steps:
    img_stats = analyze_image_statistics(img, step_name)
    print(f"{step_name:<25} {img_stats['mean']:<10.2f} {img_stats['std']:<10.2f} "
          f"{img_stats['min']:<10.2f} {img_stats['max']:<10.2f} {img_stats['entropy']:<10.4f}")

# 6. QUALITY METRICS FOR ENHANCEMENT EVALUATION
print("\n6. QUALITY METRICS FOR ENHANCEMENT EVALUATION")

def calculate_enhancement_metrics(original, enhanced):
    """Calculate various enhancement quality metrics"""

    metrics = {}

    # 1. Contrast Improvement Index (CII)
    original_contrast = np.std(original)
    enhanced_contrast = np.std(enhanced)
    metrics['CII'] = enhanced_contrast / original_contrast if original_contrast > 0 else 0

    # 2. Entropy improvement
    orig_hist, _ = np.histogram(original.flatten(), 256, [0, 256])
    enh_hist, _ = np.histogram(enhanced.flatten(), 256, [0, 256])
    orig_entropy = stats.entropy(orig_hist + 1e-10)  # Add small value to avoid log(0)
    enh_entropy = stats.entropy(enh_hist + 1e-10)
    metrics['entropy_improvement'] = enh_entropy - orig_entropy

    # 3. PSNR (if we had ground truth)
    # For demonstration, we'll calculate between original and enhanced
    mse = np.mean((original.astype(float) - enhanced.astype(float)) ** 2)
    metrics['MSE'] = mse
    metrics['PSNR'] = 10 * np.log10(255**2 / mse) if mse > 0 else float('inf')

    # 4. Visual Information Fidelity (simplified)
    metrics['info_content_ratio'] = enh_entropy / orig_entropy if orig_entropy > 0 else 0

    return metrics

# Evaluate different enhancement methods
enhancement_methods = {
    'Histogram Equalization': cv2.equalizeHist(test_img),
    'Global Stretching': contrast_stretching(test_img),
    'CLAHE': cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8)).apply(test_img),
    'Comprehensive Pipeline': pipeline_results['final']
}

print("\nEnhancement Methods Evaluation:")
print("="*80)
print(f"{'Method':<25} {'CII':<10} {'Entropy Δ':<12} {'MSE':<10} {'PSNR (dB)':<12} {'Info Ratio':<10}")
print("-"*80)

for method_name, enhanced_img in enhancement_methods.items():
    metrics = calculate_enhancement_metrics(test_img, enhanced_img)
    print(f"{method_name:<25} {metrics['CII']:<10.3f} {metrics['entropy_improvement']:<12.4f} "
          f"{metrics['MSE']:<10.3f} {metrics['PSNR']:<12.3f} {metrics['info_content_ratio']:<10.3f}")

# Visual comparison of all methods
fig, axes = plt.subplots(2, 3, figsize=(15, 10))

methods_list = list(enhancement_methods.items())
for idx in range(min(6, len(methods_list))):
    row = idx // 3
    col = idx % 3

    if idx == 0:
        axes[row, col].imshow(test_img, cmap='gray')
```

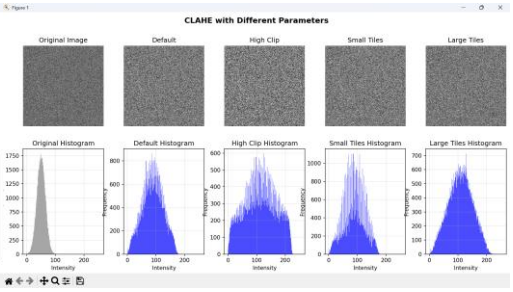
```python
    if idx == 0:
        axes[row, col].imshow(test_img, cmap='gray')
        axes[row, col].set_title('Original')
    else:
        method_name, enhanced_img = methods_list[idx-1]
        axes[row, col].imshow(enhanced_img, cmap='gray')
        axes[row, col].set_title(method_name)

    axes[row, col].axis('off')

plt.suptitle('Comparison of Different Enhancement Methods', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

print("\n=== PRAKTIKUM SELESAI ===")
print("\nRingkasan yang dipelajari:")
print("1. Point processing: negative, log, power-law transformations")
print("2. Histogram analysis and equalization techniques")
print("3. Contrast stretching and normalization methods")
print("4. Local vs global enhancement strategies")
print("5. CLAHE and comprehensive enhancement pipelines")
print("6. Quality metrics for enhancement evaluation")
```

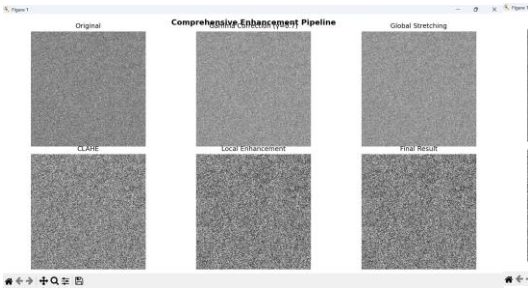
2. Screenshot Output

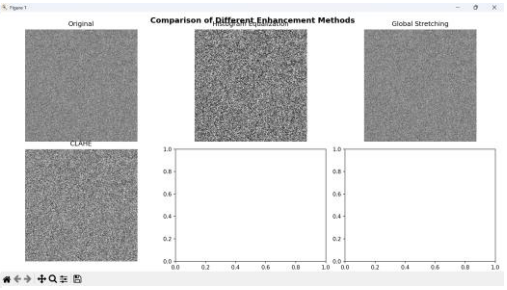










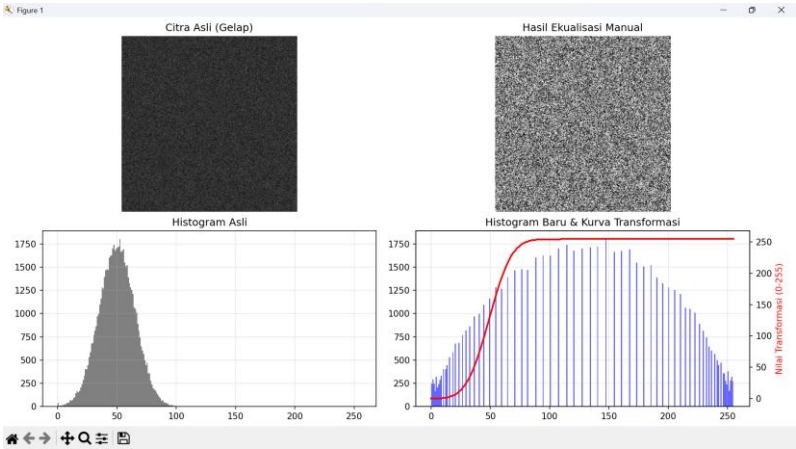


B. Program latihan1.py

1. Kode Program

```python
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.image as mpimg

def manual_histogram_equalization(image):
    """
    Manual implementation of histogram equalization
    tanpa menggunakan OpenCV sedikitpun.

    Parameters:
    image: Input grayscale image (0-255)

    Returns:
    Equalized image and transformation function
    """
    # 1. Hitung histogram menggunakan NumPy
    hist, bins = np.histogram(image.flatten(), 256, [0, 256])

    # 2. Hitung cumulative histogram (CDF)
    cdf = hist.cumsum()

    # 3. Hitung transformation function (Normalisasi)
    # Masking untuk mengabaikan nilai 0 saat mencari nilai minimum
    cdf_masked = np.ma.masked_equal(cdf, 0)

    # Rumus normalisasi: T(v) = round((cdf(v) - cdf_min) / (M*N - cdf_min) * 255)
    cdf_normalized = (cdf_masked - cdf_masked.min()) * 255 / (cdf_masked.max() - cdf_masked.min())

    # Isi kembali nilai yang di-mask dengan 0 dan ubah ke integer 8-bit (LUT)
    transformation_function = np.ma.filled(cdf_normalized, 0).astype(np.uint8)

    # 4. Apply transformation (Pemetaan pixel lama ke baru)
    equalized_image = transformation_function[image]

    # 5. Return equalized image and transformation function
    return equalized_image, transformation_function


# ==========================================
# BLOK PENGUJIAN (TANPA OPENCV)
# ==========================================
if __name__ == "__main__":

    # --- OPSI 1: Menggunakan Citra Sintetis (Dummy) ---
    # Membangun citra gelap berukuran 256x256 pixel
    test_image = np.random.normal(50, 15, (256, 256))
    test_image = np.clip(test_image, 0, 255).astype(np.uint8)

    # --- OPSI 2: Jika ingin pakai foto asli, buka komentar di bawah ini ---
    # import urllib.request
    # from PIL import Image
    # img_pil = Image.open("nama_foto_kamu.jpg").convert('L') # 'L' untuk Grayscale
    # test_image = np.array(img_pil)

    # Eksekusi fungsi manual
    eq_image_manual, transform_curve = manual_histogram_equalization(test_image)

    # Visualisasi menggunakan murni Matplotlib
    fig, axes = plt.subplots(2, 2, figsize=(12, 8))

    # Baris 1: Citra Asli vs Hasil
    axes[0, 0].imshow(test_image, cmap='gray', vmin=0, vmax=255)
    axes[0, 0].set_title("Citra Asli (Gelap)")
    axes[0, 0].axis('off')

    axes[0, 1].imshow(eq_image_manual, cmap='gray', vmin=0, vmax=255)
    axes[0, 1].set_title("Hasil Ekualisasi Manual")
    axes[0, 1].axis('off')

    # Baris 2: Histogram dan Kurva LUT
    axes[1, 0].hist(test_image.ravel(), 256, [0, 256], color='gray')
    axes[1, 0].set_title("Histogram Asli")
    axes[1, 0].grid(True, alpha=0.3)

    # Plot histogram hasil
    axes[1, 1].hist(eq_image_manual.ravel(), 256, [0, 256], color='blue', alpha=0.6, label='Histogram Baru')

    # Tambahkan Kurva Transformasi di grafik yang sama dengan sumbu Y berbeda
    ax_curve = axes[1, 1].twinx()
    ax_curve.plot(transform_curve, color='red', linewidth=2, label='Kurva LUT')
    ax_curve.set_ylabel('Nilai Transformasi (0-255)', color='red')

    axes[1, 1].set_title("Histogram Baru & Kurva Transformasi")
    axes[1, 1].grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()
```

2. Screenshot Output

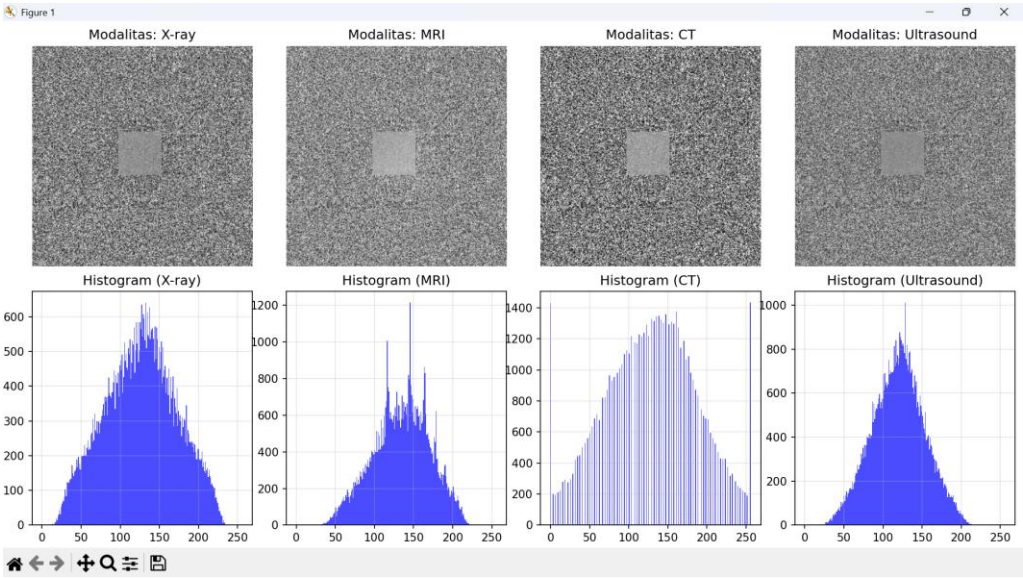


C. Program latihan2.py

1. Kode Program

```python
import cv2
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

def calculate_enhancement_metrics(original, enhanced):
    """
    Menghitung metrik kualitas untuk evaluasi peningkatan citra medis.
    """
    metrics = {}

    # 1. Contrast Improvement Index (CII)
    # Mengukur seberapa besar peningkatan kontras lokal/global
    orig_std = np.std(original)
    enh_std = np.std(enhanced)
    metrics['CII'] = enh_std / orig_std if orig_std > 0 else 0

    # 2. Entropy Improvement (Informasi Visual)
    # Mengukur jumlah informasi (detail) yang berhasil dimunculkan
    orig_hist, _ = np.histogram(original.flatten(), 256, [0, 256])
    enh_hist, _ = np.histogram(enhanced.flatten(), 256, [0, 256])

    # Ditambah 1e-10 untuk menghindari error log(0)
    orig_entropy = stats.entropy(orig_hist + 1e-10)
    enh_entropy = stats.entropy(enh_hist + 1e-10)

    metrics['Original_Entropy'] = orig_entropy
    metrics['Enhanced_Entropy'] = enh_entropy
    metrics['Entropy_Gain'] = enh_entropy - orig_entropy

    return metrics

def medical_image_enhancement(medical_image, modality='X-ray'):
    """
    Adaptive enhancement pipeline khusus untuk citra medis
    """
    # Pastikan citra format grayscale 8-bit
    if len(medical_image.shape) > 2:
        img = cv2.cvtColor(medical_image, cv2.COLOR_BGR2GRAY)
    else:
        img = medical_image.copy()

    enhanced = np.zeros_like(img)
    pipeline_info = ""

    # --- IMPLEMENTASI BERDASARKAN MODALITAS ---

    if modality == 'X-ray':
        # Constraint: Tingkatkan kontras jaringan lunak tanpa over-ekspos tulang
        # Solusi: CLAHE dengan clip limit standar
        clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
        enhanced = clahe.apply(img)
        pipeline_info = "CLAHE (clipLimit=2.0, grid=8x8)"

    elif modality == 'MRI':
        # Constraint: Angkat detail di area gelap tanpa merusak area terang
        # Solusi: Gamma correction (gamma < 1) + Mild CLAHE
        img_float = img.astype(np.float32) / 255.0
        gamma_corrected = (np.power(img_float, 0.7) * 255).astype(np.uint8)

        clahe = cv2.createCLAHE(clipLimit=1.5, tileGridSize=(8, 8))
        enhanced = clahe.apply(gamma_corrected)
        pipeline_info = "Gamma Correction (γ=0.7) -> Mild CLAHE (clip=1.5)"

    elif modality == 'CT':
        # Constraint: Fokus pada jendela spesifik, abaikan outlier (artefak logam/udara)
        # Solusi: Robust Contrast Stretching berbasis percentil
        p2, p98 = np.percentile(img, (2, 98))
        img_clipped = np.clip(img, p2, p98)

        # Normalisasi ke 0-255
        enhanced = ((img_clipped - p2) / (p98 - p2) * 255).astype(np.uint8)
        pipeline_info = "Robust Contrast Stretching (2nd-98th Percentile)"

    elif modality == 'Ultrasound':
        # Constraint: Speckle noise sangat tinggi, dilarang amplifikasi ekstrem
        # Solusi: Sangat konservatif CLAHE
        clahe = cv2.createCLAHE(clipLimit=1.0, tileGridSize=(8, 8))
        enhanced = clahe.apply(img)
        pipeline_info = "Conservative CLAHE (clipLimit=1.0) to prevent speckle amp."

    else:
        # Fallback
        enhanced = cv2.equalizeHist(img)
        pipeline_info = "Standard Global Histogram Equalization"

    # --- GENERASI REPORT ---
    metrics = calculate_enhancement_metrics(img, enhanced)

    report = {
        "Modality": modality,
        "Applied_Pipeline": pipeline_info,
        "Metrics": metrics
    }

    return enhanced, report


# ----------------------------------------
# BLOK PENGUJIAN (TESTING)
# ----------------------------------------
if __name__ == "__main__":
    # 1. Buat citra dummy yang menyerupai X-Ray (kontras rendah di tengah)
    np.random.seed(42)
    dummy_xray = np.random.normal(100, 20, (256, 256))
    dummy_xray[100:156, 100:156] = np.random.normal(150, 5, (56, 56)) # Simulasi area tulang/tumor
    dummy_xray = np.clip(dummy_xray, 0, 255).astype(np.uint8)

    # 2. Uji fungsi untuk berbagai modalitas
    modalities = ['X-ray', 'MRI', 'CT', 'Ultrasound']

    fig, axes = plt.subplots(2, 4, figsize=(18, 9))

    for idx, mod in enumerate(modalities):
        # Jalankan Adaptive Enhancement
        enh_img, report = medical_image_enhancement(dummy_xray, modality=mod)

        # Plot Citra Hasil
        axes[0, idx].imshow(enh_img, cmap='gray')
        axes[0, idx].set_title(f"Modalitas: {mod}")
        axes[0, idx].axis('off')

        # Cetak Laporan ke Terminal
        print(f"\n{'='*40}")
        print(f"REPORT: {mod.upper()}")
        print(f"Pipeline : {report['Applied_Pipeline']}")
        print(f"CII      : {report['Metrics']['CII']:.4f} (Contrast Improv. Index)")
        print(f"Ent Gain : {report['Metrics']['Entropy_Gain']:.4f} bits/pixel")

        # Plot Histogram
        axes[1, idx].hist(enh_img.ravel(), 256, [0, 256], color='blue', alpha=0.7)
        axes[1, idx].set_title(f"Histogram ({mod})")
        axes[1, idx].grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()
```

2. Screenshot Output



D. Program latihan3.py

1. Kode Program

```python
import cv2
import numpy as np
import time

class RealTimeEnhancement:
    def __init__(self, target_fps=30, buffer_size=5):
        self.target_fps = target_fps
        self.buffer_size = buffer_size

        # MEMORY CONSTRAINT: Hanya menyimpan tuple nilai min/max, BUKAN array gambar
        self.history_buffer = []

        # COMPUTATIONAL CONSTRAINT: Deklarasi objek CLAHE satu kali di memori
        # Menggunakan parameter tile 8x8 standar untuk keseimbangan speed/quality
        self.clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))

        self.prev_time = time.time()

    def enhance_frame(self, frame, enhancement_type='adaptive'):
        """
        Enhance single frame with real-time constraints
        """
        # 1. Optimasi Komputasi: Konversi ke HSV, proses HANYA kanal kecerahan (V)
        hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
        h, s, v = cv2.split(hsv)

        enhanced_v = v.copy()

        if enhancement_type == 'adaptive':
            # Ekstrak nilai ekstrem yang robust (mengabaikan noise/outlier)
            current_min = np.percentile(v, 2)
            current_max = np.percentile(v, 98)

            # Tambahkan ke buffer history
            self.history_buffer.append((current_min, current_max))

            # Batasi ukuran buffer (Memory Limit)
            if len(self.history_buffer) > self.buffer_size:
                self.history_buffer.pop(0)

            # 2. TEMPORAL CONSISTENCY: Hitung moving average untuk mencegah flickering
            smooth_min = np.mean([val[0] for val in self.history_buffer])
            smooth_max = np.mean([val[1] for val in self.history_buffer])

            # Terapkan Contrast Stretching menggunakan parameter yang sudah di-smooth
            if smooth_max > smooth_min:
                enhanced_v = ((v.astype(float) - smooth_min) / (smooth_max - smooth_min) * 255)
                enhanced_v = np.clip(enhanced_v, 0, 255).astype(np.uint8)

        elif enhancement_type == 'clahe':
            # CLAHE OpenCV sudah dioptimasi dalam C++ untuk real-time
            enhanced_v = self.clahe.apply(v)

        else:
            # Fallback: Ekualisasi global biasa (sangat berisiko flickering pada video)
            enhanced_v = cv2.equalizeHist(v)

        # 3. Gabungkan kembali kanal dan kembalikan ke format BGR
        hsv_enhanced = cv2.merge([h, s, enhanced_v])
        result_frame = cv2.cvtColor(hsv_enhanced, cv2.COLOR_HSV2BGR)

        # 4. Monitor Target FPS
        current_time = time.time()
        fps = 1 / (current_time - self.prev_time + 1e-6)
        self.prev_time = current_time

        # Visualisasi Indikator FPS pada layar
        status_color = (0, 255, 0) if fps >= (self.target_fps * 0.8) else (0, 0, 255)
        cv2.putText(result_frame, f"FPS: {fps:.1f} | Mode: {enhancement_type}",
                    (15, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.7, status_color, 2)

        return result_frame
```

```python

# ========================================
# BLOK PENGUJIAN (SIMULASI WEBCAM/VIDEO)
# ========================================
if __name__ == "__main__":
    print("Memulai simulasi real-time enhancement...")

    # Inisialisasi enhancer
    enhancer = RealTimeEnhancement(target_fps=30, buffer_size=10)

    # Buka webcam (0) atau file video ('video.mp4')
    cap = cv2.VideoCapture(0)

    if not cap.isOpened():
        print("Kamera tidak ditemukan. Menjalankan simulasi dengan noise array...")
        # (Blok ini hanya agar kode tidak error jika tidak ada webcam)
        for i in range(100):
            dummy_frame = np.random.randint(50, 150, (480, 640, 3), dtype=np.uint8)
            res = enhancer.enhance_frame(dummy_frame, enhancement_type='adaptive')
            cv2.imshow('Real-time Enhancement', res)
            if cv2.waitKey(30) & 0xFF == ord('q'): break
    else:
        print("Tekan 'q' untuk keluar, 'a' untuk Adaptive, 'c' untuk CLAHE")
        mode = 'adaptive'

        while True:
            ret, frame = cap.read()
            if not ret: break

            # Ganti mode saat tombol ditekan
            key = cv2.waitKey(1) & 0xFF
            if key == ord('q'): break
            elif key == ord('a'): mode = 'adaptive'
            elif key == ord('c'): mode = 'clahe'

            # Proses frame
            enhanced_frame = enhancer.enhance_frame(frame, enhancement_type=mode)

            # Tampilkan gambar asli berdampingan dengan hasil
            combined = np.hstack((frame, enhanced_frame))
            cv2.imshow('Kiri: Asli | Kanan: Enhanced', combined)

    cap.release()
    cv2.destroyAllWindows()
```

2. Screenshot Output